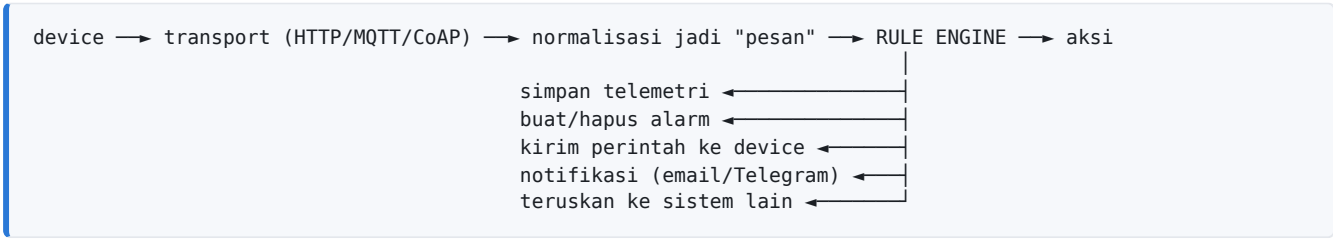
Rule Engine pada Platform IoT: Mekanisme dan Model

Catatan konsep sebelum membangun rule engine di project ini (tahap 6). Acuan utamanya ThingsBoard, AWS IoT Core, Azure IoT, Node-RED, serta mesin aturan umum seperti Drools dan mesin CEP (Complex Event Processing).

---

1. Posisi rule engine dalam platform IoT

Rule engine adalah bagian yang **memutuskan apa yang terjadi pada setiap data atau peristiwa** setelah data itu masuk, tanpa perlu menulis kode di aplikasi utama.



Di platform matang, menyimpan telemetri pun hanyalah salah satu aksi rule engine. Di ThingsBoard, misalnya, *Root Rule Chain* bawaan berisi node "Save Timeseries". Kalau node itu dihapus, data tidak tersimpan.

2. Satuan kerja: pesan (message)

Semua masukan diubah menjadi satu bentuk pesan yang seragam. Contoh, mengikuti pola ThingsBoard:

Bagian	Isi	Contoh
originator	entitas asal	Device "Greenhouse Node"
type	jenis peristiwa	`POST_TELEMETRY_REQUEST`, `POST_ATTRIBUTES_REQUEST`, `ACTIVITY_EVENT`, `INACTIVITY_EVENT`, `ALARM`
data	isi utama (JSON)	`{"soil_moisture": 18.5}`
metadata	konteks tambahan	`{"deviceName": "...", "ts": "...", "deviceType": "..."}`

Keseragaman ini penting. Peristiwa selain telemetri, seperti device terhubung atau terputus, entitas dibuat, atau alarm berubah status, ikut menjadi pesan. Akibatnya aturan bisa bereaksi pada peristiwa apa pun, tidak hanya pada nilai sensor.

3. Model-model rule engine

a. Event–Condition–Action (ECA)

Model paling dasar dan paling umum di platform yang ditujukan bagi pengguna non-programmer:

```
WHEN  telemetri baru dari device X        (event)
IF    soil_moisture < 20                   (condition)
THEN  pump = ON; kirim notifikasi          (action)
```

- **Kelebihan:** mudah dipahami dan mudah dibuat lewat form.
- **Kekurangan:** tiap aturan berdiri sendiri. Logika bertahap atau gabungan banyak sumber menjadi sulit.

Dipakai oleh fitur "automation/scene" di banyak platform smart home, dan oleh alarm rules di *device profile* ThingsBoard.

b. Flow-based / graf berarah (rule chain)

Aturan disusun sebagai **graf node** yang dihubungkan oleh relasi bernama. Ini model inti ThingsBoard dan Node-RED.

```
[Message Type Switch] ──Post telemetry──► [Script filter: msg.temp > 30] ──True──► [Create Alarm]
          │                                                  └──False──► [Clear Alarm]
          └──Post attributes──► [Save Attributes]
```

- Setiap node menerima satu pesan, mengolahnya, lalu meneruskan ke node berikutnya lewat relasi seperti `Success`, `Failure`, `True`, atau `False`.
- Kategori node di ThingsBoard:
  - **Filter:** script, message type switch, cek field.
  - **Enrichment:** menambah atribut device, customer, atau telemetri terakhir ke metadata.
  - **Transformation:** mengubah isi atau originator pesan.
  - **Action:** simpan, buat alarm, kirim RPC ke device.
  - **External:** MQTT, Kafka, REST, email.
  - **Flow:** memanggil rule chain lain.
- Kondisi rumit ditulis sebagai script kecil (JavaScript, atau TBEL di ThingsBoard) yang dijalankan di sandbox.
- **Kelebihan:** sangat fleksibel dan bisa digambar.
- **Kekurangan:** graf besar sulit dirawat dan dites. Kesalahan relasi bisa membuat pesan "hilang" diam-diam.

c. Kueri SQL atas aliran pesan

Aturan ditulis sebagai kueri yang dijalankan pada setiap pesan. Contoh di AWS IoT Core:

```
SELECT temperature, topic(2) AS device
FROM 'greenhouse/+/telemetry'
WHERE temperature > 35
```

Hasil kueri lalu dikirim ke satu atau beberapa *action*, misalnya menyimpan ke basis data, memanggil fungsi, atau menerbitkan ke topik lain. Model ini cocok untuk penyaringan dan pembentukan ulang data

per pesan, tetapi umumnya **tanpa state**: setiap pesan dinilai sendiri.

d. Complex Event Processing (CEP) dengan jendela waktu

Aturan menilai **sekumpulan peristiwa dalam rentang waktu**, bukan satu pesan. Contohnya Azure Stream Analytics, Esper, Siddhi, atau Flink CEP.

```
SELECT device, AVG(temperature) AS avg_t
FROM telemetry
GROUP BY device, TumblingWindow(minute, 5)
HAVING AVG(temperature) > 30
```

Jenis jendela	Arti
Tumbling	potongan tetap tanpa tumpang-tindih (tiap 5 menit)
Hopping/sliding	jendela 10 menit yang bergeser tiap 1 menit
Session	jendela yang berakhir setelah jeda tanpa data

CEP juga mendukung **pola urutan**, misalnya "pintu terbuka, lalu dalam 2 menit tidak ada gerakan, lalu suhu turun". Model ini kuat untuk deteksi tren dan anomali. Harganya: aturan wajib memegang state dan memikirkan semantik waktu (lihat bagian 4).

e. Production rules (forward chaining, algoritma RETE)

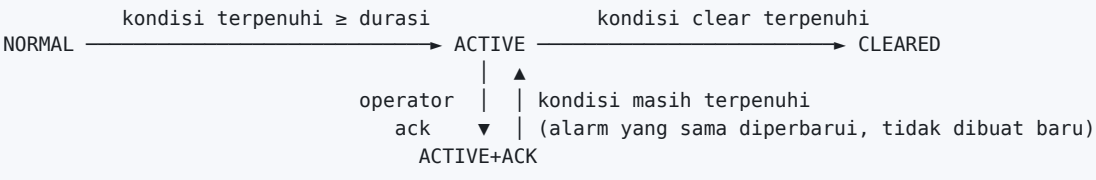
Ini model sistem pakar, misalnya Drools.

- Fakta disimpan di *working memory*, dan aturan berbentuk `when <pola atas fakta> then <aksi>`.
- Algoritma **RETE** membangun jaringan pencocokan, sehingga ketika satu fakta berubah hanya aturan yang terdampak yang dinilai ulang.
- Aksi bisa menambah fakta baru yang memicu aturan lain, yaitu *forward chaining*. Konflik antar-aturan diselesaikan dengan prioritas (*salience*).

Model ini cocok untuk ratusan aturan bisnis yang saling terkait, misalnya kebijakan energi gedung. Kekurangannya, perilaku keseluruhan sulit ditebak bila aturan saling memicu.

f. Mesin keadaan (state machine)

Model ini dipakai terutama untuk **alarm** dan status device.



- Alarm di ThingsBoard punya keparahan CRITICAL, MAJOR, MINOR, WARNING, dan INDETERMINATE.
- Statusnya adalah kombinasi aktif/clear dan acknowledged/belum.
- Satu alarm dikenali oleh pasangan *originator + jenis alarm*. Kondisi yang terus terpenuhi hanya **memperbarui** alarm yang sudah ada, tidak membanjiri dengan alarm baru.

4. Mekanisme eksekusi di balik layar

1. Stateless vs stateful.

- "suhu > 35" cukup melihat satu pesan.
- "suhu > 35 **selama 10 menit**", "**3 kali** dalam 1 jam", atau "rata-rata 5 menit" butuh state per device per aturan.
- State ini harus disimpan, di memori dengan cadangan ke DB atau Redis, supaya tidak hilang saat server restart.

2. Pencegahan flapping. Nilai yang bergoyang di sekitar ambang membuat alarm menyala-padam terus. Ada tiga teknik baku:

- **Histeresis:** menyala bila `< 20`, padam baru bila `> 25`.
- **Durasi minimum:** kondisi harus bertahan N detik.
- **Deduplikasi atau cooldown:** notifikasi paling sering sekali per 15 menit.

3. Urutan dan paralelisme.

- Pesan dari device yang sama harus diproses berurutan; kalau tidak, state "selama 10 menit" bisa kacau.
- Platform skala besar membagi antrean (Kafka atau antrean internal) berdasarkan **originator**. Satu device selalu ditangani satu partisi, sedangkan device berbeda diproses paralel.

4. Jaminan pengiriman.

- Umumnya *at-least-once*: pesan bisa diproses dua kali setelah kegagalan. Karena itu aksi harus **idempoten**, misalnya "set pump = ON" aman diulang, sedangkan "tambah 1 ke penghitung" tidak.
- ThingsBoard punya strategi *submit* (berurutan per entitas, per tenant, atau paralel penuh) dan strategi *processing* (retry gagal, abaikan gagal, retry semua) per antrean.

5. Penanganan kegagalan.

- Setiap aksi punya jalur `Failure`, misalnya email gagal terkirim.
- Pesan yang terus gagal dipindahkan ke *dead-letter queue* untuk diperiksa, bukan dibuang diam-diam.

6. Semantik waktu.

- *Event time* adalah `ts` dari device; *processing time* adalah waktu server menerima.
- Data tertunda (buffer saat Wi-Fi putus) bisa merusak jendela waktu bila memakai processing time.
- Mesin CEP memakai *watermark* dan batas *late data* untuk menangani ini.

7. Mencegah loop. Aksi "kirim perintah ke device" menghasilkan laporan baru dari device, yang bisa memicu aturan yang sama lagi. Mesin aturan perlu:

- batas kedalaman rantai,
- penanda asal pesan (*originator rule*),
- aturan yang dirancang berbasis **perubahan keadaan** ("pump belum ON"), bukan berbasis nilai saja.

8. Isolasi script.

- Script pengguna dijalankan di sandbox dengan batas waktu dan memori.

- Di ThingsBoard, JavaScript bisa dijalankan terpisah lewat layanan JS executor, sedangkan TBEL berjalan di dalam JVM.
- Tujuannya: satu script yang macet tidak menghentikan pemrosesan semua tenant.

9. Pengayaan konteks. Kondisi sering membutuhkan data lain, misalnya ambang yang disimpan sebagai atribut device ("batas kelembapan untuk tanaman ini 25%") atau nilai sensor lain di lokasi yang sama. Node *enrichment* mengambilnya sebelum kondisi dinilai, sehingga ambang bisa diubah tanpa mengubah aturan.

5. Di mana aturan dijalankan: cloud vs edge

	Rule engine di cloud	Rule engine di edge/gateway
Latensi	bergantung jaringan	milidetik
Saat internet putus	aturan tidak jalan	tetap jalan
Pandangan	seluruh armada, lintas lokasi	hanya device lokal
Contoh	ThingsBoard, AWS IoT Rules	ThingsBoard Edge, AWS IoT Greengrass, Node-RED di Raspberry Pi

Pola yang umum adalah **hibrida**:

- aturan keselamatan dan kontrol cepat (pompa, katup) di edge,
- aturan agregat, analitik, dan notifikasi di cloud,
- konfigurasi aturan disinkronkan dari cloud ke edge.

6. Ringkasan model

Model	Unit penilaian	State	Cocok untuk	Contoh
ECA	1 pesan	tidak/sedikit	otomasi sederhana	automation smart home
Flow/DAG	1 pesan melewati graf	per node, opsional	pipeline lengkap	ThingsBoard, Node-RED
SQL streaming	1 pesan	tidak	filter & routing	AWS IoT Rules
CEP + jendela	kumpulan peristiwa	ya	tren, pola urutan	Azure Stream Analytics, Esper, Flink
Production rules (RETE)	fakta di working memory	ya	banyak aturan bisnis saling terkait	Drools
State machine	transisi keadaan	ya	alarm, siklus hidup device	alarm ThingsBoard

7. Kaitannya dengan project ini

Project ini sudah punya titik masuk yang tepat. Semua telemetri (HTTP, MQTT, simulator) lewat `api/services.save_telemetry()`, yang sudah memancarkan event untuk WebSocket. Rule engine bisa dipasang di titik yang sama. Usulan desain untuk tahap berikutnya:

- **Model:** ECA dengan tambahan durasi dan histeresis. Ini cukup kaya untuk kasus nyata dan tetap mudah dibuat lewat form.
  - `Rule` berisi pemicu (variabel X), kondisi (`<`, `>`, `=`, ..., ambang, *selama* N detik), dan kondisi clear (histeresis).
  - `Action` berisi pilihan set desired state aktuator, kirim command, buat alarm, atau notifikasi.
- **Alarm** sebagai state machine (Active → Acknowledged → Cleared), dengan keparahan dan riwayat. Alarm tampil di dashboard dan didorong lewat WebSocket.
- **Eksekusi:** dievaluasi di worker terpisah (mirip `mqtt_bridge`) yang membaca antrean dari Redis. Dengan begitu penyimpanan data tetap cepat, dan aturan berdurasi bisa diperiksa berkala ("sudah 10 menit?") walaupun tidak ada data baru.
- **Contoh aturan:** "Jika `soil_moisture` < 20 selama 60 detik → set `pump` = ON; jika `soil_moisture` > 30 → set `pump` = OFF". Ini memakai desired state yang sudah ada, jadi langsung bisa diuji dengan Greenhouse Node.